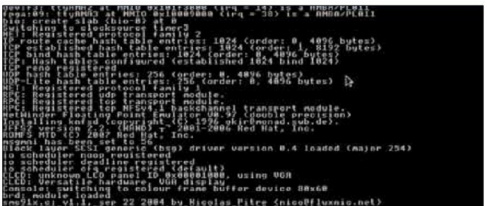

Figure 1: ARM kernel inside QEMU

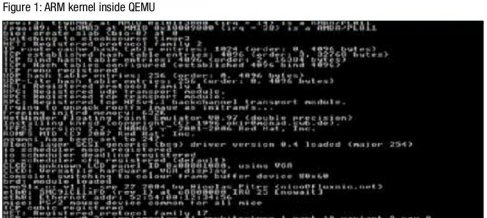

Figure 2: ARM kernel with a dummy filesystem

Create a small test program, *test.c*, with the basic 'Hello world':

```
#include<stdio.h>
int main(){
     printf("Welcome to Open World\n");
}
```

Use the ARM compiler to compile this program:

```
# arm-none-linux-gnueabi-gcc test.c -o
test
```

Once the file is compiled successfully, check the properties of the output file, showing that the output executable is built for ARM:

```
# file test
test: ELF 32-bit LSB executable, ARM,
version 1 (SYSV), dynamically linked (uses
shared libs), for GNU/Linux 2.6.16, not
stripped
```

Run the test program:

```
#qemu-arm -L /your-path/arm-2010.09/arm-
none-linux-gnueabi/libc ./test
Welcome to Open World
```

While executing the program, you must link it to the ARM library. The option *-L* is used for this purpose.

Building the Linux kernel for ARM

So, you are now done with the ARM tool-chain and *qemu-arm*. The next step is to build the Linux kernel for ARM. The mainstream Linux kernel already contains supporting files and code for ARM; you need not patch it, as you used to do some years ago. Download *linux-2.6.37.tar.bz2* from *www.kernel.org* and extract the tar ball, enter the extracted directory, and configure the kernel for ARM:

```
# tar -jxvf linux-2.6.37.tar.bz2
# cd linux-2.6.37
# make menuconfig ARCH=arm CROSS_COMPILE=arm-
none-linux-gnueabi-
```

Here, specify the architecture as ARM, and invoke the ARM tool-chain to build the kernel. In the configuration window, navigate to 'Kernel Features', and enable 'Use the ARM EABI to compile the kernel'. (EABI is Embedded Application Binary Interface.) Without this option, the kernel won't be able to load your test program.

Modified kernel for u-boot

In subsequent articles, we will be doing lots of testing on u-boot—and for that, we need a modified kernel. The kernel *zImage* files are not compatible with u-boot, so let's use *uImage* instead, which is a kernel image with the header modified to support u-boot. Compile the kernel, while electing to build a *uImage* for u-boot. Once again, specify the architecture and use the ARM tool-chain:

```
# make ARCH=arm CROSS_COMPILE=arm-none-
linux-gnueabi- uImage -s
  Generating include/generated/mach-
types.h
arch/arm/mm/alignment.c: In function 'do_
alignment':
arch/arm/mm/alignment.c:720:21: warning:
'offset.un' may be used uninitialized in
this function
```